

# Clase 109 — Vulnerabilidades de lógica de negocio

Parte: 4 — Seguridad de aplicaciones web · Fuente: *Real-World Bug Hunting (Yaworski) / PortSwigger*

 Duración estimada: 100 min · Nivel: Avanzado

## Objetivo

Descubrir **fallos de lógica de negocio**: vulnerabilidades que no rompen la tecnología sino las reglas de la aplicación (precios negativos, saltarse pasos, abusar de descuentos, condiciones de carrera). No las detectan los escáneres automáticos; requieren entender el flujo y pensar como un adversario creativo.

 **Ética:** solo en labs propios/autorizados. Manipular transacciones reales es fraude.

## Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Modelar** el flujo de negocio para encontrar suposiciones frágiles.
2. **Explotar** fallos de validación de precios, cantidades y estados.
3. **Saltar** pasos de un flujo multi-etapa (flow bypass).
4. **Detectar** y explotar condiciones de carrera (race conditions).
5. **Recomendar** validaciones de negocio del lado servidor.

## Temas

| # | Tema                                | Por qué importa                    |
|---|-------------------------------------|------------------------------------|
| 1 | Qué es una falla de lógica          | Categoría distinta de las técnicas |
| 2 | Suposiciones del desarrollador      | Donde vive el bug                  |
| 3 | Manipulación de precios/cantidades  | Impacto económico directo          |
| 4 | Flow bypass                         | Saltar validaciones                |
| 5 | Race conditions                     | Estado inconsistente               |
| 6 | Abuso de descuentos/cupones         | Casos reales frecuentes            |
| 7 | Defensa: validar reglas en servidor | Cierre del fallo                   |

## Definiciones y características

- **Falla de lógica:** violación de una regla de negocio, no de una tecnología. Característica: invisible para los escáneres.

- **Flow bypass:** completar una operación saltándose pasos o validaciones intermedias. Característica: explota confianza en el orden del flujo.
- **Race condition (TOCTOU):** dos operaciones concurrentes producen un estado inconsistente. Característica: ventana entre comprobación y uso.
- **Manipulación de parámetros de negocio:** alterar precio, cantidad o estado en la petición. Característica: el servidor debe recalcular, no confiar.
- **Idempotencia:** una operación repetida no cambia el resultado. Característica: su ausencia habilita abusos (doble canje).
- **Límite lógico:** restricción de negocio (máximos, mínimos). Característica: si no se valida en servidor, se evade.

## Herramientas y preparación

- **PortSwigger labs** de business logic y race conditions.
- **Juice Shop** (varios retos de lógica).
- **Burp** (Repeater, Intruder y el modo de peticiones paralelas para race conditions).

## Laboratorio guiado

 Solo en labs propios.

1. Mapea un flujo de compra completo: carrito → checkout → pago → confirmación.
2. Intercepta el checkout y **manipula el precio** o la cantidad (valores negativos, decimales, cero).
3. Prueba un **flow bypass**: envía la petición de confirmación sin completar el pago.
4. Abusa de un cupón: aplícalo varias veces o combínalo indebidamente.
5. Explota una **race condition**: envía peticiones paralelas de canje/retiro con Burp (single-packet attack) para superar un límite.
6. Comprueba límites lógicos: compra más unidades de las permitidas alterando el parámetro.
7. Documenta la regla violada, la petición manipulada y el impacto económico.

## Ejercicios

1. Encuentra 3 suposiciones del desarrollador que se puedan romper en Juice Shop.
2. Manipula un precio a un valor negativo y explica el impacto.
3. Diseña un flow bypass saltando un paso de validación.
4. Reproduce una race condition de canje múltiple con peticiones paralelas.
5. Explica por qué recalcular en servidor evita la manipulación de precios.
6. Propón validaciones de negocio para un carrito de compra.

## Reto verificable

Resuelve un lab de lógica de negocio de PortSwigger (manipulación de precio, flow bypass o race condition) y demuestra el beneficio indebido. **Criterio de aceptación:** el lab queda resuelto, documentas la regla de negocio vulnerada, la petición manipulada/paralela y la defensa (validación y recálculo en servidor, idempotencia, bloqueos).

## Errores comunes

| Síntoma / mensaje           | Causa y cómo arreglar                            |
|-----------------------------|--------------------------------------------------|
| El precio se recalcula      | Servidor valida; busca otro parámetro de negocio |
| Flow no se puede saltar     | Estados bien controlados; documenta la fortaleza |
| Race condition no reproduce | Envía peticiones más simultáneas (single-packet) |
| Cupón no reutilizable       | Idempotencia correcta; prueba combinaciones      |
| Escáner no encontró nada    | Normal: la lógica requiere análisis manual       |

## Preguntas frecuentes

 **¿Por qué los escáneres no las detectan?** Porque dependen del significado de negocio, que la herramienta no entiende. Requieren razonamiento humano.

 **¿Qué es una race condition en la práctica?** Aprovechar la ventana entre "comprobar" y "usar" enviando peticiones simultáneas para, por ejemplo, canjear dos veces un mismo saldo.

 **¿Cómo se defienden?** Validando y recalculando toda regla en el servidor, usando bloqueos/transacciones e idempotencia en operaciones sensibles.

## Referencias

- Yaworski, *Real-World Bug Hunting*.
- PortSwigger Business logic vulnerabilities: <https://portswigger.net/web-security/logic-flaws>
- PortSwigger Race conditions: <https://portswigger.net/web-security/race-conditions>
- OWASP WSTG — Business Logic Testing.

## Siguiente clase

Clase 110 - Seguridad de APIs REST